Activation Functions in Deep Learning | Sigmoid, Tanh & ReLU

➢ Introduction:

In a neural network, each neuron:

1. Takes multiple **inputs**,
2. Multiplies them by **weights**,
3. Adds a **bias**,
4. And then passes this value through an **activation function (AF)** to produce an output.

$$y = f(Wx + b)$$

Here, (f) = activation function.

➢ What are Activation Functions?

Activation Functions decide **whether a neuron should "fire" or stay inactive** based on its input.

They introduce **non-linearity** into the network.

Without them, the network would act like a simple linear regression, no matter how many layers you add.

➢ Importance of Activation Functions

Activation functions allow neural networks to:

- Learn **complex, non-linear patterns** (like images, language, speech).
- Capture **relationships** that are not straight lines.
- Transform weighted sums into **useful outputs** (probabilities, signals, etc.)

Without them → your deep network is just a big *linear equation*.

🤔 Why Are Activation Functions Needed?

Because real-world problems (like speech, images, or language) are **non-linear**.

If we remove activation functions:

$$Output = W_2(W_1x)$$

→ Still linear, no matter how many layers we add.

With activation functions:

$$Output = f_2(W_2(f_1(W_1x)))$$

→ Non-linear transformations → more expressive power.

➢ Ideal Activation Function (What We Want)

An ideal activation function should:

1. Be **non-linear**
2. Be **differentiable** (for backpropagation)
3. Have **small computation cost**
4. Avoid **vanishing or exploding gradients**
5. Allow **faster convergence**

➢ Sigmoid Activation Function

$$f(x) = \frac{1}{1 + e^{-x}}$$

- Converts any input into a value between **0 and 1**
- Often used for **binary classification** (output layer)

Input (x)	Output (f(x))
-∞	0
0	0.5
+∞	1

➢ Advantages

- Smooth and continuous
- Good for **probability outputs**
- Historically popular (used in early neural networks)

➢ Disadvantages

- **Vanishing gradient** for large or small values of x
- **Not zero-centered** → slows learning
- Computation is **expensive (uses exponent)**

➢ Tanh (Hyperbolic Tangent) Activation Function

$$f(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$

- Outputs between **-1 and 1**
- S-shaped curve (like Sigmoid, but zero-centered)

Input (x)	Output (f(x))
-∞	-1
0	0
+∞	1

➢ Advantages

- **Zero-centered output** → faster convergence
- Stronger gradients than sigmoid
- Works better for hidden layers than sigmoid

➢ Disadvantages

- Still suffers from **vanishing gradient**
- Computationally expensive (exponentials involved)

➢ ReLU (Rectified Linear Unit) Activation Function

$$f(x) = \max(0, x)$$

If input < 0 → output = 0
If input ≥ 0 → output = x

➢ Advantages

- Very **fast computation**
- Solves **vanishing gradient problem** (mostly)
- Works well with **deep networks**
- Sparse activation → efficient (some neurons inactive)

➢ Disadvantages

- **Dying ReLU problem:** neurons permanently output 0 for negative inputs
- Not zero-centered

Visual Intuition (Summary)

Function	Formula	Range	Non-linear?	Used In
Sigmoid	1 / (1 + e^-x)	(0, 1)	✅	Binary output layer
Tanh	(e^x - e^-x)/(e^x + e^-x)	(-1, 1)	✅	Hidden layers
ReLU	max(0, x)	[0, ∞)	✅	Deep networks, CNNs

➢ Choosing the Right Activation Function

Situation	Recommended Function
Binary Classification Output	Sigmoid
Hidden Layers	ReLU or Leaky ReLU
Data Centered Around 0	Tanh
Deep Networks (Speed Priority)	ReLU
Avoid Vanishing Gradient	ReLU / Leaky ReLU / ELU

➢ Outro / Summary

Key Idea	Explanation
Purpose	Introduce non-linearity in neural networks
Main Functions	Sigmoid, Tanh, ReLU
Best Practice	Use ReLU in hidden layers, Sigmoid in output (for binary), Softmax for multi-class
Caution	Watch out for vanishing gradient (Sigmoid/Tanh) and dying ReLU

➢ Simple Definition

Activation Functions decide how strongly a neuron should respond to its inputs — they bring "intelligence" to a neural network by introducing non-linearity.